CROPER · PRUEBA TÉCNICA

Desarrollador Full Stack

Respuestas teóricas: arquitectura, rendimiento y seguridad · Uso de IA

1. Ejercicio 3 – Arquitectura de microservicios
2. Ejercicio 4 – Optimización de rendimiento web en móviles
3. Ejercicio 5 – Seguridad en APIs y cloud
4. Uso de inteligencia artificial

Repositorio: https://github.com/crhistian27/prueba-tecnica-croper

Código: API NestJS + MongoDB y frontend Angular + NgRx (ver README del repositorio).

Fecha: 7 de noviembre de 2026

Ejercicio 3 – Arquitectura de microservicios

> **Supuestos.** Diseño para una plataforma tipo marketplace de insumos agropecuarios (fertilizantes, semillas, agroquímicos, herramientas, nutrición animal) en Colombia, con varios vendedores (distribuidores y casas comerciales) y compradores que son principalmente productores rurales. Los supuestos de negocio están explícitos para que las decisiones se puedan discutir.

---

1. Lo que el contexto agro le exige a la arquitectura

Realidad del negocio	Consecuencia técnica
Conectividad rural intermitente (3G/4G inestable, celulares de gama media)	Clientes *offline-first* (PWA), APIs **idempotentes** para reintentar sin duplicar pedidos, respuestas pequeñas y paginadas
Estacionalidad fuerte: picos de demanda en siembra y fertilización, coordinados con las temporadas de lluvia	Escalado horizontal por servicio y **colas** que absorban picos sin perder pedidos
Pago contraentrega, PSE y crédito o financiación a productores	Pagos como dominio aislado y auditable; pedidos con estados intermedios ("pendiente de pago al entregar")
Logística de última milla a veredas, con transportadoras regionales y tiempos variables	Integración con varios operadores; seguimiento basado en eventos, no en consultas constantes
Productos regulados (agroquímicos con registro ICA, restricciones de venta y transporte)	Reglas de catálogo y validaciones en el checkout por producto y región
Precio y disponibilidad por región (bodegas, fletes)	Inventario por bodega y precios calculados según el destino
Canales que usa el productor: WhatsApp, SMS, llamadas	Servicio de notificaciones multicanal desacoplado del resto

2. Decisión principal: microservicios sí, pero no desde el día uno

Recomendación: empezar con un **monolito modular** (lo que ya implementé en NestJS: módulos `auth`, `users` y `products` con límites claros) y **extraer servicios de forma gradual** (*strangler fig*) cuando haya una razón concreta:

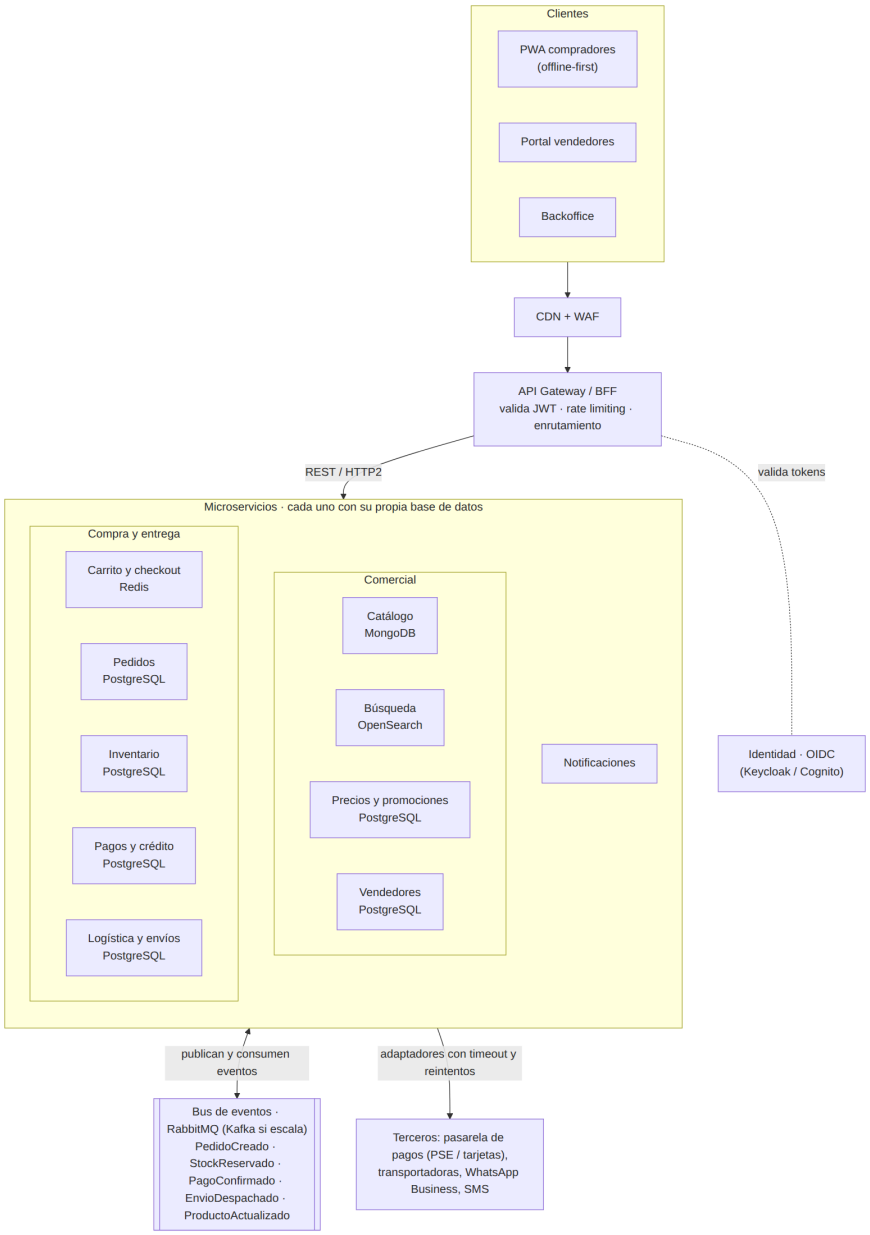
- un módulo necesita **escalar distinto** (búsqueda en temporada alta),
- tiene **requisitos de cumplimiento propios** (pagos),
- integra **terceros inestables** que no deben tumbar el resto (transportadoras, pasarelas), o

- lo mantiene **un equipo distinto** que necesita desplegar de forma independiente.

Por qué: los microservicios cambian complejidad de código por complejidad operativa (red, consistencia eventual, observabilidad distribuida, despliegues coordinados). Con un equipo pequeño, eso frena más de lo que ayuda. El diseño de abajo es la **arquitectura objetivo**, y el plan de evolución (sección 9) dice en qué orden llegar a ella.

---

3. Arquitectura objetivo



Servicios y por qué cada uno es un límite

Servicio	Responsabilidad	Datos	Por qué separarlo
Identidad	Login, registro, roles (comprador, vendedor, admin, soporte), MFA	Proveedor OIDC gestionado	Se compra, no se construye: es crítico y estándar
Catálogo	Productos, categorías, atributos distintos por tipo (la fórmula NPK de un fertilizante frente a las medidas de una herramienta), registro ICA, fichas técnicas	**MongoDB**: esquema flexible por categoría	Muchas lecturas y pocas escrituras; esquema heterogéneo
Búsqueda	Texto libre, sinónimos regionales ("abono" ↔ "fertilizante", "urea" ↔ "nitrógeno"), tolerancia a errores de escritura, filtros	**OpenSearch**, alimentado por eventos	Carga y tecnología distintas; escala independiente en temporada
Inventario	Existencias por bodega, **reservas con expiración**	**PostgreSQL** (transacciones, bloqueo optimista)	La consistencia de stock es crítica; evita la sobreventa en picos
Precios y promociones	Precio por región y volumen, descuentos de temporada, cupones	PostgreSQL / Redis	Cambia seguido; reglas de negocio propias
Carrito y checkout	Carrito persistente entre dispositivos y orquestación de la compra	**Redis** (carrito)	Estado efímero y alto tráfico; inicia la saga de compra
Pedidos	Ciclo de vida del pedido y fuente de verdad de su estado	**PostgreSQL**	Núcleo del negocio; auditoría y reportes
Pagos y crédito	Contraentrega, PSE y tarjeta vía pasarela, crédito y cupo del productor, conciliación	**PostgreSQL** aislado	Cumplimiento (PCI vía tokenización, auditoría); aislar el riesgo
Logística y envíos	Cotización de fletes por municipio o vereda, guías, seguimiento, integración con transportadoras	PostgreSQL	Integra terceros lentos o inestables; no debe bloquear la compra
Vendedores	Onboarding, catálogo propio, liquidaciones y comisiones	PostgreSQL	Dominio de marketplace con su propio portal
Notificaciones	WhatsApp, SMS, email, push; plantillas y preferencias	Cola + log de envíos	Solo consume eventos; reintentos propios sin afectar a nadie

4. Comunicación entre servicios

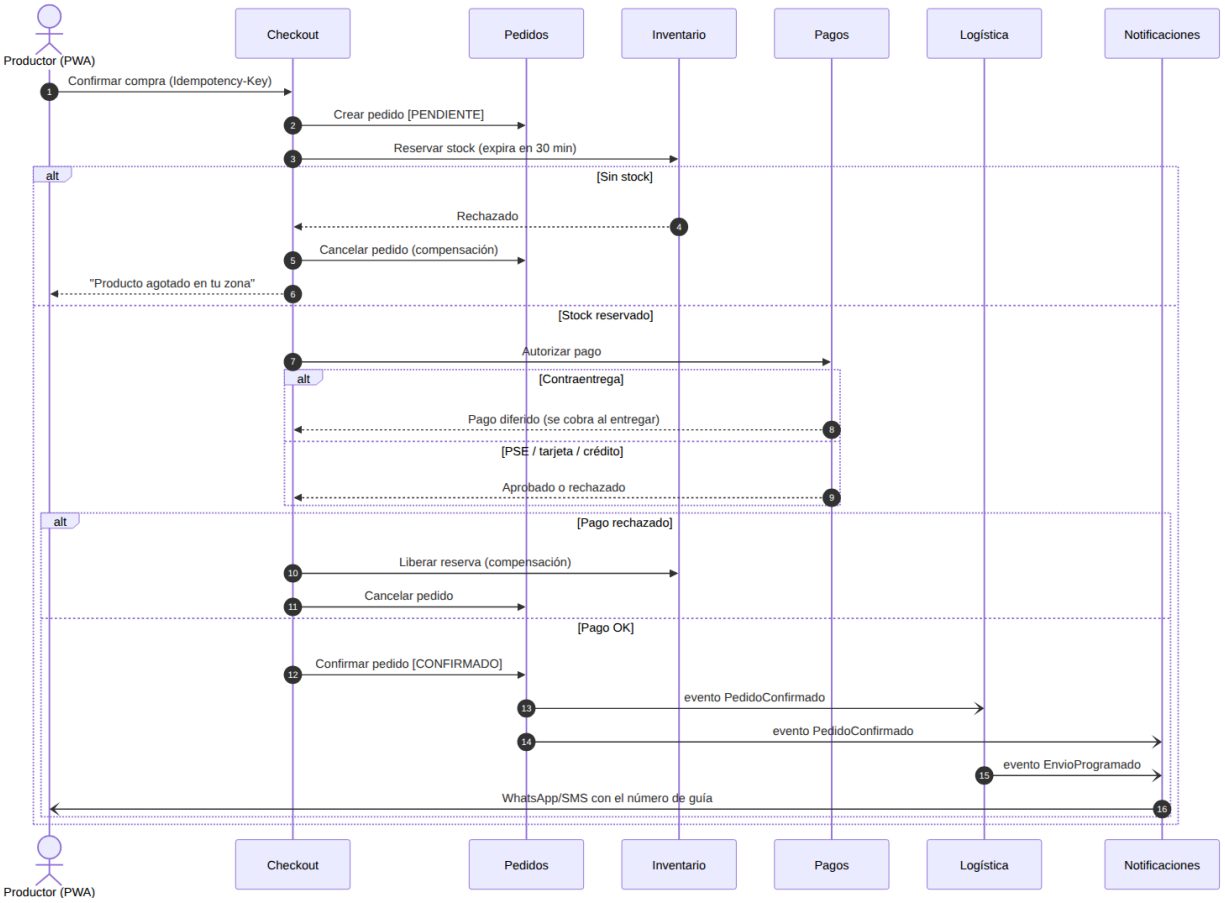
Regla general: síncrono para consultar, asíncrono para reaccionar.

Tipo	Cuándo	Tecnología
Síncrona: cliente → gateway	Todo lo que pide un usuario	REST/JSON (HTTP/2), paginado y comprimido
Síncrona: servicio → servicio	Consultas que necesitan respuesta inmediata (checkout pregunta precio y stock)	REST o **gRPC** interno, siempre con **timeout, reintentos con backoff y circuit breaker**
Asíncrona: eventos	Hechos de negocio que interesan a otros: `PedidoCreado`, `StockReservado`, `PagoConfirmado`, `EnvioDespachado`, `ProductoActualizado`	Bus de eventos con colas durables y *dead-letter queue*

Broker: empezaría con **RabbitMQ** (o SQS/SNS gestionado), más simple de operar. Pasaría a **Kafka** solo si se necesita reprocesar el histórico de eventos (analítica, nuevos consumidores que "rebobinan") o con volúmenes muy altos.

Flujo crítico: la compra como saga orquestada

Una compra toca inventario, pagos y logística, y **no puede haber una transacción distribuida** entre sus bases de datos. Se usa el patrón **Saga**: cada paso es una transacción local con su **compensación** si un paso posterior falla.



Garantías que hacen confiable el flujo con mala conectividad:

- **Idempotencia:** el cliente envía un `Idempotency-Key` por compra. Si la conexión se cae y la PWA reintenta, no se crea un segundo pedido ni un segundo cobro.
- **Transactional outbox:** cada servicio guarda el cambio de estado y el evento a publicar en la **misma transacción local**, y un proceso los publica después. Así nunca queda "pedido confirmado pero evento perdido".
- **Consumidores idempotentes** (guardan los IDs de eventos procesados): el bus puede entregar un evento más de una vez.
- **Orquestación vs. coreografía:** el checkout se **orquesta** (un coordinador explícito; el flujo es crítico y debe ser fácil de seguir y depurar). Las reacciones secundarias (notificar, indexar en búsqueda, analítica) se hacen por **coreografía** (cada servicio escucha eventos), lo que permite agregar consumidores sin tocar el flujo principal.

---

5. Datos

- **Una base de datos por servicio:** ningún servicio lee las tablas de otro. Si necesita datos ajenos, los pide por API o mantiene una **copia local alimentada por eventos** (por ejemplo, Búsqueda indexa lo que publica Catálogo).
- **Políglota según el caso:** documentos (catálogo heterogéneo), relacional (dinero, pedidos y stock, donde manda la consistencia), búsqueda (texto con sinónimos) y clave-valor (carrito y caché).
- **Consistencia eventual explícita en la UX:** "Tu pedido está siendo confirmado" en lugar de prometer algo que aún no es definitivo.
- **Reportes y analítica** (ventas por región y temporada, demanda para planificar inventario) en un **data warehouse** alimentado por eventos, no consultando las bases operacionales.

---

6. Seguridad

Se apoya en las respuestas del Ejercicio 5:

- **Identidad centralizada con OIDC.** El gateway valida el JWT (firma, expiración, audiencia) y **cada servicio vuelve a autorizar** (*zero trust*): estar dentro de la red no da acceso.
- **Autorización por rol y por recurso:** un vendedor solo edita *sus* productos y ve *sus* pedidos (verificación de propiedad, no solo de rol).
- **Servicio a servicio: mTLS** (service mesh) e identidad de carga de trabajo; sin llaves de larga duración.
- **Pagos:** tarjetas **tokenizadas por la pasarela** (los datos de tarjeta nunca tocan nuestros servidores, lo que reduce el alcance PCI), webhooks firmados y conciliación diaria.
- **Datos personales:** cumplimiento de la **Ley 1581 de 2012 (Habeas Data)**: consentimiento, finalidad y acceso mínimo a cédula, dirección y teléfono; cifrado en reposo y enmascarado en logs.
- **Borde:** WAF, rate limiting por usuario e IP en el gateway, protección contra bots en el checkout y los cupones.
- **Secretos** en un gestor con rotación; **cadena de suministro** con escaneo de dependencias e imágenes en CI.

---

7. Operación y observabilidad

- **Trazas distribuidas (OpenTelemetry)** con un ID de correlación que viaja por HTTP y por los eventos: permite seguir una compra a través de 6 servicios.
- **Logs estructurados centralizados, métricas y SLOs** por servicio (ej. checkout: 99,9 % de éxito, p95 < 800 ms) con alertas.
- **Despliegue:** contenedores en un orquestador **gestionado** (EKS/GKE o, más simple, ECS / Cloud Run) con **autoescalado** por CPU y por longitud de cola, preparado para los picos de siembra.

- **CI/CD independiente por servicio** con **contract testing** (Pact) para no romper a los consumidores de una API o de un evento; despliegues *canary* y *feature flags*.
- **Resiliencia:** timeouts en toda llamada, *bulkheads* (un proveedor de SMS lento no agota las conexiones del checkout) y degradación elegante (si Búsqueda cae, se navega por categorías desde Catálogo).

---

8. Trade-offs

Decisión	Ganamos	Pagamos	Mitigación
Microservicios vs. monolito	Escalado y despliegue independientes, aislamiento de fallos	Latencia de red, consistencia eventual, más infraestructura y monitoreo	Empezar modular y extraer solo con una razón concreta
Base de datos por servicio	Autonomía, tecnología adecuada a cada caso	Sin JOINs entre dominios, datos duplicados	Copias locales por eventos; warehouse para reportes
Asíncrono con eventos	Desacople, absorbe picos, tolera caídas	Más difícil de depurar, entregas duplicadas o desordenadas	Outbox, idempotencia, trazas con correlación
Saga orquestada	Flujo visible y testeable	El orquestador concentra lógica	Mantenerlo delgado: solo coordina, las reglas viven en cada servicio
RabbitMQ antes que Kafka	Operación simple, ideal para comandos y colas	Sin *replay* nativo del histórico	Migrar si aparecen casos de reprocesamiento o analítica en streaming
Servicios gestionados (IdP, colas, BD)	Menos operación, seguridad por defecto	Costo y dependencia del proveedor	Interfaces propias (puertos y adaptadores) para poder cambiar de proveedor

---

9. Plan de evolución

1. **Fase 0 – Monolito modular (hoy).** Módulos con límites claros, como en el proyecto actual: `ProductsModule` es el futuro Catálogo y `AuthModule` se reemplaza por el proveedor OIDC. Eventos *internos* entre módulos para practicar el desacople.
2. **Fase 1 – Extraer lo de menor acoplamiento y mayor beneficio: Notificaciones** (solo consume eventos) y **Búsqueda** (escala distinto en temporada). Introducir el bus de eventos y el outbox.
3. **Fase 2 – Pagos** (aislar el cumplimiento y el riesgo) y **Logística** (aislar las integraciones con terceros inestables).
4. **Fase 3 – Pedidos e Inventario con la saga de checkout**, cuando el volumen o los equipos lo justifiquen.

Cada extracción se hace detrás del gateway (*strangler fig*): el cliente no se entera, y se puede revertir si algo sale mal.

Ejercicio 4 – Optimización de rendimiento web en móviles

> Las técnicas no son teóricas: se aplicaron sobre la app de los ejercicios 1 y 2 y se **midieron** con Lighthouse en perfil móvil. Al final se explica cómo medí y qué aprendí del proceso de medición.

Contexto: por qué importa en Croper

Los usuarios de un e-commerce agropecuario suelen conectarse desde **zonas rurales con 3G/4G inestable**, con **celulares de gama media o baja** y **planes de datos limitados**. Cada KB y cada viaje de red cuesta tiempo y dinero. Por eso las métricas que más importan son:

- **LCP** (*Largest Contentful Paint*): cuándo aparece el contenido principal.
- **FCP** (*First Contentful Paint*): cuándo el usuario deja de ver una pantalla en blanco.
- **INP / TBT**: qué tan rápido responde la página a toques y clics.
- **CLS**: que el contenido no "salte" mientras carga.
- **Peso transferido**: los datos que consume el plan del usuario.

Resultados medidos

Perfil móvil de Lighthouse 13 (Moto G Power, 4G lento, CPU 4× más lenta), **caché vacía** (primera visita) y **throttling real** (`devtools`). Mediana de 3 ejecuciones; la dispersión entre ejecuciones fue menor a 0,1 s.

Métrica	Login antes	Login después	Productos antes	Productos después
FCP (primer contenido visible)	4,0 s	**0,7 s**	5,1 s	**0,7 s**
LCP (contenido principal)	4,0 s	3,9 s	5,1 s	**4,4 s**
Speed Index	4,0 s	3,7 s	4,1 s	**3,1 s**
Peso transferido	447 KB	**245 KB (−45 %)**	507 KB	**304 KB (−40 %)**
CLS	0	0	0,002	0,002
Accesibilidad	100	100	96	**100**
Recursos que bloquean el render	2	**0**	2	**0**

Además, en el Ejercicio 2 el **bundle inicial bajó de 691 kB a 351 kB** (161 → 98 kB comprimido) gracias a la carga diferida. Hoy es de 386 kB (106 kB comprimido) tras el rediseño visual (la configuración global de íconos importa `MatIcon` en el bundle inicial).

Técnicas aplicadas

1. Dividir y reducir el JavaScript (carga diferida)

Qué es: enviar solo el código necesario para la pantalla actual y cargar el resto bajo demanda.

Cómo se aplicó:

- **Rutas diferidas** (`loadComponent` / `loadChildren`): el login no descarga el código de la tabla, el store de productos ni los componentes de Material que solo usa el panel.
- **Estado de NgRx por feature** (`provideState` / `provideEffects` en la ruta): el store y los effects de productos viajan en el chunk diferido, no en el bundle inicial.
- **Importación dinámica de lo que no se necesita al inicio:** `MatSnackBar` , y con él todo el sistema de *overlay* del CDK, se carga con `import()` la primera vez que se muestra un mensaje.
- **Detección del problema midiendo, no adivinando:** con *sourcemaps* se descubrió que una sola línea (`MatPaginatorIntl` importado en la configuración global) arrastraba paginator → select → overlay → forms al bundle inicial.
- **Acortar la cadena de peticiones:** el router cargaba el layout y *después* descubría el chunk de productos, es decir, dos viajes de red seguidos antes de poder pedir datos a la API. Al agrupar el área autenticada en un solo chunk (`core/layout/authenticated.routes.ts`) se eliminó un nivel de la cadena: **LCP de productos 5,1 s → 4,4 s**.

Métricas que mejora: LCP, TBT/INP (menos JS que parsear en CPUs lentas) y datos consumidos.

2. Fuentes e íconos optimizados y auto-alojados

Qué es: las fuentes suelen ser de los recursos más pesados y, si vienen de otro dominio, bloquean el render.

Cómo se aplicó:

- **Subconjunto de íconos:** la fuente completa de Material Icons pesaba **170 KB** (más que todo el JS inicial) para usar ~40 de sus 2.000+ íconos. `scripts/build-icon-font.mjs` genera una fuente **solo con los íconos usados: 5,9 KB (−96 %)**. El script falla si una plantilla usa un ícono que no está en la lista, así nunca falta un ícono en producción.
- **Auto-alojamiento:** Inter se sirve desde nuestro dominio (`@fontsource-variable/inter`), una sola fuente variable para todos los pesos, y el navegador descarga solo el subconjunto *latin* gracias a `unicode-range` . Se eliminaron las 2 hojas de estilo de Google Fonts que **bloqueaban el render** y la conexión extra a otros dominios (DNS + TLS, muy caros en 3G).
- Se eliminó una fuente monoespaciada (21 KB) usada solo para el SKU; ahora se usa la del sistema.
- `font-display: swap` para el texto (se lee de inmediato con la fuente del sistema) y `block` para íconos (evita mostrar la palabra "delete" en lugar del ícono).

Métricas que mejora: peso transferido (−200 KB por página), FCP y Speed Index.

3. Percepción de velocidad: app shell, esqueletos y estabilidad visual

Qué es: el usuario tolera mejor la espera si ve progreso inmediato y la pantalla no salta.

Cómo se aplicó:

- **App shell en `index.html`:** HTML y CSS mínimos en línea (~1 KB) que pintan la marca y "Cargando panel…" **antes** de que llegue el JavaScript. **FCP de 4,0–5,1 s a 0,7 s**: el usuario ve una respuesta 3+ segundos antes en lugar de una pantalla en blanco.
- **Esqueletos (skeletons)** con la forma real de la tabla y de las tarjetas de indicadores mientras llegan los datos, en lugar de un spinner genérico.
- **CLS ≈ 0:** la barra de progreso tiene altura reservada, los esqueletos ocupan el mismo espacio que el contenido final y la tabla se atenúa en lugar de desaparecer al paginar.
- **Respuestas inmediatas a la interacción:** botones deshabilitados con spinner al guardar, `exhaustMap` para evitar envíos dobles con conexiones lentas y el diálogo que no se cierra si falla (no se pierde lo escrito).

Métricas que mejora: FCP, CLS y la experiencia percibida.

4. Menos datos por la red: paginación, búsqueda eficiente y caché HTTP

Cómo se aplicó:

- **Paginación, filtros y orden del lado del servidor:** cada página de 10 productos pesa **~2 KB**. Nunca se descarga el catálogo completo, algo clave cuando crezca a miles de productos.
- **Indicadores calculados en MongoDB** con una agregación (`GET /products/stats` , < 1 KB) en lugar de traer todos los productos para sumarlos en el celular.
- **Búsqueda con `debounceTime(400)`** y mínimo 2 caracteres, más `switchMap` , que **cancela la petición anterior** si el usuario sigue escribiendo o paginando.
- **nginx:** compresión gzip; JS, CSS y fuentes con hash en el nombre se cachean **1 año como `immutable`** (en la segunda visita no se descargan), e `index.html` con `no-cache` para recibir siempre la última versión.

Métricas que mejora: datos consumidos, LCP en visitas repetidas e INP.

5. Render eficiente y UX táctil

- **Angular *zoneless* + signals + `ChangeDetectionStrategy.OnPush`:** solo se re-renderiza lo que cambió; los componentes leen el store con `selectSignal` .
- **`trackBy` / `@for track`** en listas: al paginar no se recrean filas que no cambiaron.
- **Diseño móvil específico, no solo "encogido":** en pantallas < 600 px la tabla se reemplaza por tarjetas, el botón "Nuevo producto" pasa a ser flotante al alcance del pulgar, las áreas táctiles miden ≥ 44 px y los campos usan `inputmode="numeric"` / `"email"` para mostrar el teclado adecuado.
- **Accesibilidad:** contraste mínimo 4,5:1 (se corrigió un gris de 3,09:1 detectado por Lighthouse: **96 → 100**), respeto a `prefers-reduced-motion` , `aria-label` en botones de ícono y foco visible.

Decisiones y trade-offs

- **El puntaje de Lighthouse no es el objetivo.** Con el app shell, el puntaje de productos bajó de 76 a 71 porque el TBT subió (50 → 540 ms). La causa: TBT solo cuenta las tareas largas que ocurren *después* del primer pintado, y como ahora se pinta mucho antes, el arranque de Angular cae dentro de esa ventana. **El JavaScript no hace más trabajo**; el usuario ve contenido 3,6 s antes. Se priorizó la experiencia real sobre el número.
- **SEO 63 a propósito:** `robots.txt` bloquea la indexación porque es un **panel de administración privado**. En la tienda pública sería al revés: SSR o prerender para SEO.
- **El LCP sigue en ~4 s en 4G lento** porque la app se renderiza en el navegador: el contenido real depende de descargar y ejecutar JS y luego llamar a la API. El siguiente paso para bajarlo está en las propuestas.

Propuestas para el siguiente paso (no implementadas)

1. **PWA con Service Worker (`@angular/pwa`):** la más importante para el contexto rural.
   - Cachear la app (las visitas siguientes cargan sin red).
   - Cachear el catálogo con estrategia *stale-while-revalidate*: se muestran datos aunque falle la conexión.
   - Encolar altas y ediciones sin conexión con *Background Sync* y enviarlas al recuperarla.
2. **SSR o prerender (Angular SSR)** en la tienda pública: el HTML llega con contenido, lo que baja el LCP y mejora el SEO. En el panel privado, el app shell da gran parte del beneficio sin un servidor Node.
3. **Imágenes de producto** (cuando existan): `NgOptimizedImage`, WebP/AVIF, `srcset` por tamaño de pantalla, `loading="lazy"` bajo el pliegue, `priority` en la imagen LCP y un CDN de imágenes que redimensione.
4. **Listas largas:** *virtual scroll* del CDK si el catálogo se muestra sin paginar (por ejemplo, un selector de productos en un pedido).
5. **Brotli y HTTP/2–3 detrás de un CDN** (~15–20 % menos que gzip y mejor multiplexación en redes con pérdida de paquetes).
6. **Respetar el ahorro de datos:** con `Save-Data` / `navigator.connection.saveData`, evitar imágenes pesadas o precargas.
7. **Medición continua:**
   - **Lighthouse CI** en cada PR con presupuestos (*budgets*) que fallen el build si el bundle o el LCP empeoran.
   - **RUM** (`web-vitals` → analítica) para medir a usuarios reales en campo, porque el laboratorio no reproduce la red de una vereda.

Metodología de medición (y una lección aprendida)

- Script reproducible: `frontend/scripts/lighthouse-audit.mjs`. Usa Lighthouse y puppeteer-core, vacía la caché HTTP antes de cada medición e inyecta la sesión del usuario demo para medir `/products`.

- **Comparación A/B en igualdad de condiciones:** la versión anterior se construyó desde el commit previo y se levantó en otro puerto contra la misma API; se alternaron las ejecuciones de "antes" y "después".
- **Lección:** la primera medición dio "0 KB transferidos" porque reutilizaba la caché del navegador, y la simulación de Lighthouse (`simulate`) dio resultados **bimodales** en esta app (LCP de 0,6 s o 2,2 s según la ejecución). Una sola ejecución habría llevado a conclusiones falsas. Por eso los resultados finales usan **caché vacía, throttling real (`devtools`) y la mediana de varias ejecuciones**.

Ejercicio 5 – Seguridad en APIs y cloud

> Cada respuesta se apoya en lo implementado en los ejercicios 1 y 2. Las referencias a archivos indican dónde verlo en el código.

---

1. OAuth vs JWT

No son alternativas: resuelven problemas distintos y se usan juntos con frecuencia.

	OAuth 2.0 / OpenID Connect	JWT
Qué es	Un **protocolo de autorización** (y OIDC, una capa de autenticación sobre él)	Un **formato de token**: JSON firmado (`header.payload.firma`)
Responde a	¿Cómo obtiene una aplicación permiso para actuar en nombre de un usuario, sin ver su contraseña?	¿Cómo transporto afirmaciones (quién es, qué puede hacer, hasta cuándo) de forma verificable?
Define	Flujos (Authorization Code + PKCE, Client Credentials…), roles (cliente, servidor de autorización, recurso), scopes, refresh tokens	Estructura, firma (HS256, RS256…) y claims estándar (`sub` , `exp` , `iat` , `aud`)
Ejemplo	"Iniciar sesión con Google", una app de terceros que lee pedidos de un vendedor	El `accessToken` que devuelve nuestro `POST /auth/login`

Un servidor OAuth puede emitir sus access tokens **en formato JWT**, y es lo más común.

Cuándo usar cada uno:

- **JWT propio (lo que implementé):** una sola aplicación con su propio login, sin terceros. Es simple y *stateless*: la API valida la firma sin consultar la base de datos.
- **OAuth 2.0 / OIDC con un proveedor de identidad** (Keycloak, Auth0, Cognito): varios clientes (web, app móvil, integraciones de distribuidores), login social, SSO entre servicios o terceros que acceden a la API. En una arquitectura de microservicios (Ejercicio 3) es la opción recomendada, porque centraliza la identidad en un solo lugar.

Buenas prácticas aplicadas o a considerar:

- Access token de vida corta (`JWT_EXPIRES_IN=3600` s) y secreto de ≥ 32 caracteres, validado al arrancar (`backend/src/config/env.validation.ts`).
- Algoritmo fijado al validar (`algorithms: ['HS256']` en `jwt.strategy.ts`), lo que evita ataques de confusión de algoritmo (`alg: none`).
- El JWT **no es cifrado**, solo firmado: el payload se lee en Base64. Por eso solo lleva `sub` y `email` , nunca datos sensibles.

- **Limitación conocida:** un JWT no se puede revocar antes de que expire. En producción usaría un access token corto (5–15 min) más un **refresh token rotativo en cookie** `httpOnly; Secure; SameSite`, con una lista de revocación (Redis) para el logout y el robo de sesión.

---

2. Protección contra fuerza bruta

Ninguna medida basta sola; la defensa se hace en capas:

1. **Rate limiting.** *Implementado* con `@nestjs/throttler`: 5 intentos por minuto por IP en `/auth/login` y `/auth/register`, y 100 por minuto en el resto de la API (`auth.controller.ts`, `app.module.ts`). Al superarlo la API responde `429` y el frontend muestra "Demasiados intentos. Espera un minuto".
   - *Detalle corregido en esta revisión:* detrás de nginx, la API veía a **todos** los usuarios con la IP del proxy, así que 5 fallos de cualquiera bloqueaban a todos. Se configuró `trust proxy` solo para el primer salto (`TRUST_PROXY=1`) y se dejó el puerto de la API accesible solo en `localhost`. Verificado: un cliente bloqueado no puede saltarse el límite enviando un `X-Forwarded-For` falso, y otro cliente sigue pudiendo intentar.
2. **Hash lento de contraseñas.** *Implementado:* bcrypt con 12 rondas (`users.service.ts`). Aunque se filtre la base de datos, probar contraseñas offline es muy costoso. Argon2id sería la alternativa más moderna.
3. **Sin pistas para el atacante.** *Implementado:* mensaje genérico "Email o contraseña incorrectos". Si el email no existe, igual se compara contra un hash ficticio para que el tiempo de respuesta sea el mismo y no se puedan enumerar usuarios (`auth.service.ts`).
4. **Política de contraseñas.** *Implementado:* mínimo 8 caracteres, con letras y números, en el registro.
5. **En producción agregaría:**
   - **Límite por cuenta además de por IP**, con backoff exponencial (1, 2, 4, 8… min) tras N fallos. Así se frenan ataques distribuidos desde muchas IPs contra un mismo usuario.
   - **Contador compartido en Redis** (`ThrottlerStorage`): el almacenamiento actual está en memoria y no funciona con varias réplicas de la API.
   - **CAPTCHA** solo después de varios fallos, para no afectar a usuarios legítimos.
   - **MFA** para cuentas administrativas.
   - **WAF / rate limiting en el borde** (Cloudflare, AWS WAF) y **alertas** ante picos de 401 o 429.

---

3. Seguridad en bases de datos MongoDB

Riesgo	Medida	Estado en el proyecto
Acceso sin autenticación (causa de muchas filtraciones masivas)	Autenticación siempre habilitada	✅ Mongo arranca con usuario y clave (`MONGO_INITDB_ROOT_*`)

Riesgo	Medida	Estado en el proyecto
Exposición a internet	Red privada; nunca IP pública ni `0.0.0.0`	✅ Puerto publicado solo en `127.0.0.1`; la API se conecta por la red interna de Docker
Inyección NoSQL (`{"email": {"$gt": ""}}`)	Validar tipos y rechazar campos desconocidos	✅ DTOs con `class-validator` y `whitelist` + `forbidNonWhitelisted`. Verificado: ambos payloads devuelven `400`
ReDoS / inyección en búsquedas con regex	Escapar la entrada del usuario	✅ `escapeRegex()` en la búsqueda de productos (con test)
Exposición de datos sensibles	Proyecciones; no devolver campos internos	✅ `password` con `select: false` y eliminado del JSON
Privilegios excesivos	Usuario de aplicación con mínimo privilegio	✅ La API usa `croper_app`, con rol `readWrite` **solo** sobre `croper`, creado por `mongo/init/01-create-app-user.js`. Verificado: no puede leer otras bases, ver o crear usuarios, borrar la base ni apagar el servidor. El usuario root queda solo para administración
Tráfico interceptado	TLS entre la API y Mongo (`tls=true`)	⚠️ No aplica en local; obligatorio en la nube
Robo de discos o backups	Cifrado en reposo y backups cifrados	⚠️ Lo ofrece MongoDB Atlas o el disco cifrado del proveedor
Pérdida de datos	Backups automáticos con restauraciones *probadas*; réplica de 3 nodos	⚠️ Infraestructura
Actividad sospechosa	Auditoría, alertas y versiones actualizadas	⚠️ Infraestructura

Recomendación: usar **MongoDB Atlas** con *Private Endpoint / VPC Peering* (sin IP pública), usuarios por servicio con mínimo privilegio, TLS obligatorio, cifrado en reposo, backups *point-in-time* y la cadena de conexión guardada en un gestor de secretos.

---

4. Despliegue de microservicios en la nube

Principio general: **defensa en profundidad y mínimo privilegio en cada capa.**

Red

- VPC con **subredes privadas** para servicios y bases de datos; solo el balanceador o API Gateway queda en la subred pública.

- **HTTPS en todo el tráfico externo** (certificados gestionados, HSTS) y **mTLS entre servicios** (service mesh como Istio o Linkerd), para que un servicio comprometido no pueda hacerse pasar por otro.
- *Security groups* / *NetworkPolicies* de Kubernetes que solo permitan los flujos necesarios (por ejemplo, Pedidos → Inventario, y no Catálogo → Pagos).
- **WAF** delante del gateway: OWASP Top 10, bots, rate limiting global.

Identidad y acceso

- **API Gateway** que valida el JWT u OAuth en el borde, más validación en cada servicio (*zero trust*).
- **IAM con roles por servicio** (IRSA en EKS, Workload Identity en GKE): sin llaves de larga duración en el código.

Secretos

- En un gestor de secretos (AWS Secrets Manager, Vault, GCP Secret Manager) con **rotación**, inyectados en tiempo de ejecución. Nunca en el repositorio ni en la imagen.
- *En el proyecto:* `.env` en `.gitignore`, `.env.example` sin valores reales, `.dockerignore` excluye `.env` y la API no arranca si falta el secreto o es débil.

Contenedores y cadena de suministro

- Imágenes mínimas y **usuario no root**. *En el proyecto:* build multi-etapa, solo dependencias de producción y `USER node` en la API.
- **CI/CD con controles automáticos:** `npm audit` / Dependabot, escaneo de imágenes (Trivy) y de secretos (gitleaks), SAST. Imágenes firmadas (cosign) y solo se despliegan si pasan los controles.
- Sistema de archivos de solo lectura, *resource limits* y *Pod Security Standards*.

Operación

- Logs centralizados **sin datos sensibles** (tokens, contraseñas), trazas distribuidas (OpenTelemetry) y alertas de seguridad.
- Cabeceras HTTP de seguridad. *En el proyecto:* `helmet` en la API y `X-Frame-Options`, `X-Content-Type-Options` y `Referrer-Policy` en nginx.
- En producción: **deshabilitar o proteger Swagger**, CORS restringido a los dominios reales y CSP también en el frontend.

Resumen: implementado vs. pendiente para producción

Implementado y verificado	Pendiente para producción
JWT con expiración, algoritmo fijado y secreto validado	Refresh token en cookie `httpOnly` y revocación
Rate limiting por IP real detrás de nginx (5/min en login)	Límite por cuenta y contador compartido en Redis

Implementado y verificado	Pendiente para producción
bcrypt (12), mensajes genéricos, tiempo constante	MFA para administradores
Validación estricta (anti NoSQL injection), regex escapado	TLS entre la API y Mongo
Usuario de Mongo con mínimo privilegio (`readWrite` solo en su base)	Usuario de solo lectura para reportes
Mongo y API sin exposición pública; usuario no root en la API	Gestor de secretos, WAF, mTLS, escaneo en CI
helmet, cabeceras en nginx, CORS restringido	Swagger protegido o deshabilitado; CSP en el frontend

Uso de inteligencia artificial

Herramientas utilizadas

Herramienta	Uso
Claude Code (Anthropic)	Análisis de la prueba, planificación, generación y revisión de código

Enfoque

- **Decisiones mías:** el stack del frontend (Angular + NgRx), usar Docker, el orden de los ejercicios, pedir el rediseño visual, los botones para copiar las credenciales demo y corregir el usuario de MongoDB antes de continuar.
- **Trabajo de la IA:** generó el código, los tests y los borradores de los documentos.
- **Regla de trabajo:** nada se dio por bueno sin evidencia: tests automatizados, pruebas en el navegador, `curl` contra la API en ejecución y mediciones con Lighthouse.
- **Errores encontrados en el camino:** varios eran errores de la propia IA o de herramientas generadas (ver la tabla final). Se corrigieron y se documentaron, en lugar de ocultarlos.

Registro de prompts relevantes

Formato de cada entrada: qué se pidió, qué generó la IA, cómo se validó y qué se corrigió.

1. Análisis y planificación

- **Prompt:** "Léeme el PDF, analízamelo y créame una planificación paso a paso para responder cada uno de estos puntos."
- **Resultado:** análisis por ejercicio, stack propuesto y plan por pasos con tiempos estimados.
- **Ajuste propio:** se definió Angular + NgRx para el frontend (decisión propia, no de la IA).

2. Preparación del repositorio (Paso 0)

- **Prompt:** "Comienza con el paso 0" (con Docker).
- **Resultado:** estructura del monorepo, `docker-compose.yml` con MongoDB, `.gitignore`, `.env.example`, README base.
- **Validación:** se levantó MongoDB con `docker compose up -d` y se verificó el healthcheck.
- **Decisiones revisadas:** puerto de Mongo expuesto solo en `127.0.0.1`; `.env` excluido del repositorio.

3. Backend NestJS + MongoDB (Paso 1)

- **Prompt:** "Paso 1" (siguiendo el plan: CRUD de productos, validaciones, Swagger, JWT, tests, Docker).
- **Resultado:** módulos `products`, `auth`, `users`, `health`; filtro global de errores; seed; tests unitarios y e2e; Dockerfile multi-etapa.
- **Validación:**
  - Pruebas manuales con `curl` de cada endpoint y cada caso de error (401, 400, 404, 409, 429).
  - 8 tests unitarios + 12 tests e2e contra MongoDB real en una base de datos aislada.
  - Arranque en Docker verificado con healthcheck.

4. Frontend Angular + NgRx (Paso 2)

- **Prompt:** "Paso 2" (siguiendo el plan: listar/crear/editar/eliminar, tabla paginada, errores amigables, State Management, responsive).
- **Resultado:** app Angular 22 con features `auth` y `products`, store NgRx, interceptors, guards, diálogos y vista móvil; Dockerfile con nginx.
- **Validación:**
  - Prueba manual en el navegador: login con clave incorrecta y correcta, búsqueda, creación con SKU duplicado (409) y válido, eliminación con confirmación, estado "sin resultados", paginación.
  - 16 tests unitarios (reducer de productos y mapeo de errores HTTP).
  - Verificación de cabeceras HTTP de nginx con `curl`.

5. Rediseño visual

- **Prompt:** "Revisa y mejora el diseño, se ve muy básico, quiero que se vea más profesional."
- **Resultado:** layout tipo panel con menú lateral, tarjetas de indicadores (nuevo endpoint `GET /products/stats`), tabla con íconos por categoría y estado de stock, login en dos columnas, diálogo por secciones y sistema de diseño propio.
- **Validación:** revisión visual en el navegador en escritorio y en un iframe de 390 px para la vista móvil; test e2e del nuevo endpoint.

6. Ejercicio 5 – Seguridad

- **Prompt:** "Sí, el 5."
- **Resultado:** `docs/ejercicio-5-seguridad.md`, con cada respuesta ligada a lo implementado y una lista honesta de lo pendiente para producción.
- **Validación:** antes de afirmar cada medida se verificó en el código (grep) y contra la API en ejecución (payloads de inyección NoSQL → 400, prueba de rate limiting con dos clientes distintos).

7. Ejercicio 4 – Rendimiento

- **Prompt:** "Paso 4."

- **Resultado:** se midió la app con Lighthouse en perfil móvil, se aplicaron las optimizaciones que la medición señaló (subconjunto de íconos, fuentes auto-alojadas, cadena de chunks más corta, app shell, contraste) y se documentó en `docs/ejercicio-4-rendimiento.md`.
- **Validación:** comparación A/B contra la versión anterior (construida desde el commit previo), con caché vacía, throttling real y mediana de varias ejecuciones; revisión visual de los íconos tras cambiar la fuente.

8. Ejercicio 3 – Arquitectura de microservicios

- **Prompt:** "Paso 3."
- **Resultado:** `docs/ejercicio-3-arquitectura.md`: requisitos derivados del contexto agro, recomendación de evolución gradual desde un monolito modular, 11 servicios con su justificación, comunicación (saga orquestada, outbox, idempotencia), datos, seguridad, operación, trade-offs y plan por fases, con 2 diagramas Mermaid.
- **Validación y ajustes propios:**
  - Los diagramas se renderizaron con mermaid-cli para validar la sintaxis **y revisarlos visualmente**. El primero era técnicamente válido pero ilegible (flechas cruzadas y etiquetas superpuestas), así que se rediseñó por capas.
  - Los supuestos de negocio quedaron explícitos en lugar de presentarse como hechos sobre la empresa.

9. Entrega (Paso 6)

- **Prompt:** "Sí, seguimos" (README final, PDF y revisión antes de entregar).
- **Resultado:**
  - README como portada del proyecto, con capturas generadas automáticamente.
  - PDF generado desde los mismos Markdown con `docs/tools/build-pdf.mjs`.
- **Validación:**
  - Se clonó el repositorio en una carpeta limpia y se levantó solo con `cp .env.example .env` y `docker compose up -d`: frontend, Swagger, login demo, 32 productos y usuario de MongoDB con mínimo privilegio.
  - Se revisó el historial de git buscando secretos reales (ninguno).
  - Se revisó el PDF página por página: se corrigieron una página casi vacía antes del diagrama, un enlace sobrante y un símbolo mal renderizado.

Correcciones y validaciones destacadas

Problema detectado	Causa	Corrección
La API no arrancaba: `mongoose does not provide an export named 'Connection'`	El proyecto es ESM (NestJS 12). Con `emitDecoratorMetadata`, los tipos `Model` / `Connection` de los constructores se convertían en imports reales de un paquete CommonJS	Importarlos con `import type`: el token de inyección ya lo dan `@InjectModel` / `@InjectConnection`

Problema detectado	Causa	Corrección
Los tests fallaban: `fertilizantes is not a valid type at path Fertilizantes`	Vitest transpila con otro compilador que emite el enum como tipo de la propiedad en vez de `String`	Declarar `type` explícito en todos los `@Prop`, sin depender de la metadata del compilador
Logs del seed no aparecían	El contexto de Nest se creó con nivel de log `error/warn`, que silenciaba `Logger.log`	Usar `console.log` en el script de seed
Un comando `pkill -f "node dist/main.js"` mató la propia shell	El patrón coincidía con la línea de comando del shell que lo ejecutaba	Patrón `[n]ode dist/main.js`
El scaffold de Nest generó un `AGENTS.md` con publicidad de un servicio de telemetría	Archivo generado por el CLI	Ignorado como instrucción y eliminado del repo
Bundle inicial de 691 kB (superaba el presupuesto de 500 kB)	`MatPaginatorIntl` importado en `app.config.ts` y en `app.routes.ts` arrastraba paginator → select → overlay → forms al bundle inicial	Medición con sourcemaps; mover la configuración de la feature a `products.routes.ts` cargado con `loadChildren` y cargar `MatSnackBar` con `import()` → **351 kB**
Las cabeceras de seguridad de nginx no llegaban al navegador	En nginx, un `add_header` dentro de un `location` anula los heredados del `server`	Detectado revisando las respuestas con `curl -I`; cabeceras movidas a un archivo incluido en cada `location`
El mensaje de SKU duplicado decía "Ya existe un registro con ese sku"	Mensaje genérico construido con el nombre técnico del campo	Mensajes específicos por campo en el filtro de errores del backend
El símbolo `$` del campo precio se superponía a la etiqueta	`matTextPrefix` visible antes de que la etiqueta flote	Eliminado el prefijo (la etiqueta ya indica "COP")
El build fallaba al no poder descargar Google Fonts	Angular intenta incrustar el CSS de las fuentes en producción, lo que hace depender el build de internet	Desactivar `optimization.fonts` para que el build (y el de Docker) sea reproducible
Rango de paginación duplicado y filtro de categoría truncado	Detectado en la revisión visual	Ocultar el rango del paginador y ampliar el filtro

Problema detectado	Causa	Corrección
Detrás de nginx, 5 logins fallidos de **cualquier** usuario bloqueaban el login de **todos**	Express veía siempre la IP del proxy, así que el rate limiting se aplicaba a una sola IP compartida	`trust proxy` configurable (`TRUST_PROXY=1`) y puerto de la API solo en `localhost` . Verificado: un cliente bloqueado no se libera falsificando `X-Forwarded-For` y otro cliente no queda afectado
La API se conectaba a MongoDB con el usuario root	Configuración inicial por simplicidad, detectada al responder el Ejercicio 5	Usuario `croper_app` con `readWrite` solo en su base, creado por un script idempotente de `docker-entrypoint-initdb.d` y aplicado sin borrar datos. Los tests e2e limpian documentos en vez de `dropDatabase` . Verificado: 7 operaciones probadas (2 permitidas, 5 bloqueadas) y una instalación desde cero en un contenedor temporal
La primera medición de Lighthouse daba resultados demasiado buenos ("0 KB transferidos")	El navegador reutilizaba la caché porque la página se había abierto antes	Vaciar la caché HTTP (CDP) antes de cada medición y fijar la sesión desde `/api/health` para no precargar la app
Con la simulación de Lighthouse, el LCP salía 0,6 s o 2,2 s según la ejecución	La simulación (`simulate`) estima tiempos a partir de una carga rápida y daba resultados bimodales	Comparar con throttling real (`devtools`) y la mediana de varias ejecuciones; las conclusiones cambiaron (el login no mejoraba el LCP solo con bajar peso)
El LCP del login no bajaba aunque el peso bajó un 45 %	En una app renderizada en el navegador el camino crítico es el JS, no las fuentes	App shell en `index.html` : FCP 4,0 → 0,7 s. Se documentó el trade-off (sube el TBT medido)
La fuente de íconos pesaba 170 KB	Se cargaba la familia completa (2.000+ íconos) para usar ~40	Subconjunto generado por script (5,9 KB) que además valida que las plantillas no usen íconos fuera de la lista